7

Building classes

This chapter demonstrates how to create and use virtual objects for Object Oriented Programming with PHP.

Encapsulating data

A class is a data structure that can contain both variables and functions in a single entity. These are collectively known as its “members” – the variables are known as its class “properties” and the functions are known as its class “methods”.

Access to class members from outside the class is controlled by “access specifiers” in the class declaration. Typically, these will deny access to the class variables but allow access to methods that can store and retrieve data from those variable members. This technique of “data hiding” ensures that stored data is safely encapsulated within the class variable members, and is the first principle of Object Oriented Programming (OOP).

A class declaration begins with the **class** keyword, followed by a space, then a programmer-specified name – adhering to the usual PHP naming conventions but beginning in uppercase. Next come the class property declarations followed by the class method declarations. These are like regular variable and function declarations but should each begin with an access specifier. The class declaration syntax looks like this:

```
class ClassName
{
  access-specifier $var1 , $var2 , $var3 ;

  access-specifier functionName1 ( $arg1 , $arg2 , $arg3 )
  {
    statements-to-execute ;
  }
}
```

It is conventional to begin class names with an uppercase character and object names with lowercase.

An access specifier may be any one of the keywords **public**, **private** or **protected**, to specify access rights for its listed members:

- **public** members are accessible from any place where the class is visible.
- **private** members are accessible only to other members of the same class in which they are defined.
- **protected** members are accessible only to other members of the same class and members of classes derived from that class.

Derived classes are introduced later in this chapter – see here.

Any real-world object can be defined by its attributes and by its actions. For example, a dog has attributes such as age, weight and color, and actions it can perform such as bark. The class mechanism in PHP provides a way to create a virtual dog object within a script, where the class properties can represent its attributes and the class methods represent its actions:

```
class Dog
{
  private $age ;
  private $weight ;
  private $color ;

  public function bark() { echo 'WOOF!' ; }

  // Plus methods here to store data in the properties.

  // Plus methods here to retrieve data from the properties.
}
```

While a PHP script class cannot perfectly emulate a real-word object, the aim is to encapsulate all relevant attributes and actions as class properties and methods.

It is important to recognize that a class declaration only defines a data structure – in order to create an object you must declare an "instance" of that data structure. This is achieved by assigning a copy of a class to a variable using the **new** keyword, like this:

```
$fido = new Dog() ;      // Creates an instance named "fido"
                         // of the programmer-defined
                         // "Dog" data structure.
```

The instance object is a working copy of the original class data structure so retains all its features. As the method in the example above has been declared **public**, it is visible in the script outside the class declaration. This means the instance's copy of the method can be called using a special **->** object operator, like this:

```
$fido->bark() ;          // Outputs WOOF!
```

The **->** object operator can be used to reference any visible class member, but the variable properties in the example above have been declared **private** so are encapsulated in the object and are not directly accessible. The principle of encapsulation in PHP programming describes the grouping together of data and functionality in class members – these are the age, weight, color attributes and bark action in this Dog class.

Notice that you must add parentheses after the class name in the statement creating an instance object.

Creating an object

In order to assign and retrieve data from private members of a class, special **public** accessor methods must be added to the class. These are “setter” methods to assign data, and “getter” methods to retrieve data. Accessor methods are often named as the variable they address, with the first letter made uppercase, and prefixed by “set” or “get” respectively. For example, accessor methods to address an **age** variable may be named **setAge()** and **getAge()**.

The setter and getter methods can reference a member of the same class using a special **$this** “pseudo-variable”, followed by the **->** object operator, and the property name. For example, using **$this->age** to reference an age property:

1. Begin a PHP script with a statement to declare a class named “Dog”

```
<?php
class Dog
{
  // Members to be inserted here (Steps 2-5).
}
```

object.php

2. Between the braces of the Dog class declaration, declare three private variable members

```
private $age ;
private $weight ;
private $color ;
```

3. After the private variables, add a public method to output a string when called

```
public function bark() { echo ‘WOOF! WOOF! <br>’ ; }
```

4 Add public setter methods – to assign individual argument values to each of the private variables

```
public function setAge ( int $yrs )
{ $this->age = $yrs ; }

public function setWeight ( int $lbs )
{ $this->weight = $lbs ; }

public function setColor ( string $fur )
{ $this->color = $fur ; }
```

In the class declaration, notice that all methods are declared public and all variables are declared private. This notion of “public interface, private data” is a key concept when creating classes.

5 Add public getter methods – to retrieve individual values from each of the private variables

```
public function getAge() { return $this->age ; }

public function getWeight() { return $this->weight ; }

public function getColor() { return $this->color ; }
```

Fido

6 After the Dog class declaration, declare an instance of the Dog class named “fido”

```
$fido = new Dog() ;
```

7 Add statements calling each setter method to assign data

```
$fido->setAge( 3 ) ;
$fido->setWeight( 15 ) ;
$fido->setColor( ‘brown’ ) ;
```

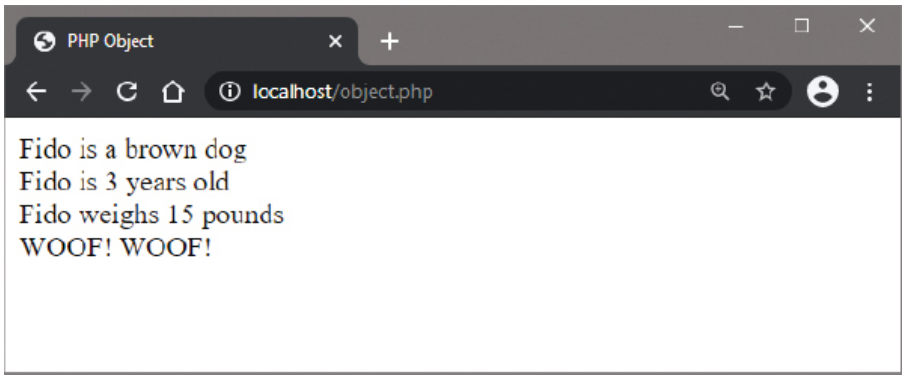
8 Add statements calling each getter method to retrieve the assigned values

```
echo 'Fido is a '.$fido->getColor().' dog <br>' ;
echo 'Fido is '.$fido->getAge().' years old <br>' ;
echo 'Fido weighs '.$fido->getWeight().' pounds<br>' ;
```

9 Now, add a call to the regular output method

```
$fido->bark() ;
?>
```

10 Save the document in your web server's **/htdocs** directory as **object.php** then open the page via HTTP to see the output class properties and method call



This **object.php** script will get modified over the next few pages as new features are incorporated.

Initializing members

A program can easily create multiple objects simply by declaring multiple instances of a class, and each object can have unique attributes by assigning individual values with its setter methods.

It is often convenient to combine the setter methods into a single method that accepts arguments for each private variable. This means that all values can be assigned with a single statement in the program, but the method will contain multiple statements.

Optionally, each parameter can specify a permissible data type for each argument being passed, and may also usefully specify default values to each parameter:

1. Rename a copy of the previous example “object.php” as a new program “initializer.php”

initializer.php

2. In the Dog class declaration, replace the three setter methods with a single combined setter method that specifies the argument data types and default values

```
public function setValues
( int $yrs = 2 , int $lbs = 8 , string $fur = ‘black’ )
{
  // Statements to be inserted here.
}
```

3. In the method definition block, insert three statements to assign values from passed arguments to class variables

```
  $this->age = $yrs ;
  $this->weight = $lbs ;
  $this->color = $fur ;
```

4. After the modified class definition, replace the calls to the three setter methods by a single call to the combined setter method – passing three arguments

```
$fido->setValues( 3 , 15 , 'brown' ) ;
```

5. Declare a second instance of the Dog class named "pooch"

```
$pooch = new Dog() ;
```

6. Add a second call to the combined setter method – passing three arguments for the new object

```
$pooch->setValues( 4, 18, 'gray' ) ;
```

Try passing an incorrect data type to the initializer method to see a type error occur.

7. Add statements calling each getter method to retrieve the assigned values

```
echo '<hr>Pooch: '.$pooch->getAge().'yrs' ;
echo $pooch->getWeight().'lbs '.$pooch->getColor() ;
```

8. Add a second call to the regular output method

```
$pooch->bark() ;
```

9. Now, declare a third instance of the Dog class named "rover"

```
$rover = new Dog() ;
```

10. Add a third call to the combined setter method – this time passing no arguments for the new object

```
$rover->setValues( ) ;
```

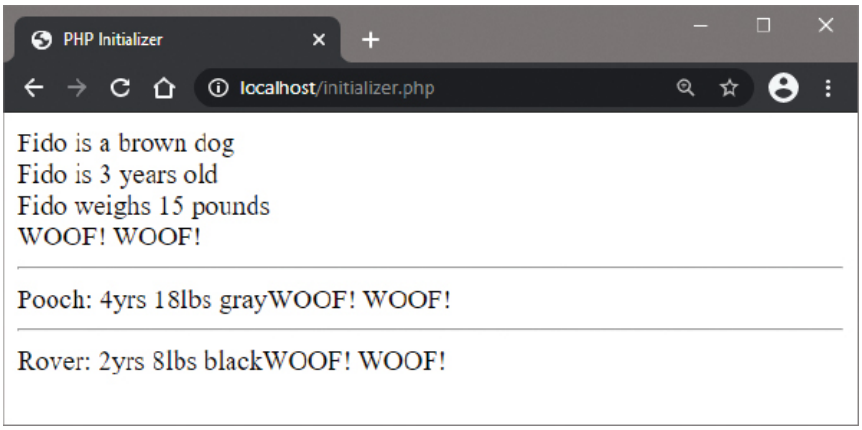
11. Add statements calling each getter method to retrieve the assigned values, which are specified default values

```
echo '<hr>Rover: '.$rover->getAge().'yrs' ;
echo $rover->getWeight().'lbs '.$rover->getColor() ;
```

12 Add a third call to the regular output method

```
$rover->bark() ;
```

13 Save the document in your web server's **/htdocs** directory as **initializer.php** then open the page via HTTP to see the output set by the combined initializer method



You must still call the initializer method here, to assign the default values to the class variables when passing no arguments.

Using constructors

Class variable members can be initialized by a special “constructor” method that is called whenever an instance of the class is created. The constructor method is always named **__construct()** and can accept arguments to set the initial value of class variables.

A constructor method has a companion “destructor” method – that is called whenever an instance of the class is destroyed. The destructor method is always named **__destruct()**.

Constructor and destructor methods have no return value and are called automatically – they cannot be called explicitly, and consequently their declarations need no access specifier.

Values to initialize class variables are passed to the constructor method in the statement creating an object, in parentheses following the object name:

1. Rename a copy of the previous example “initializer.php” as a new program “constructor.php”

constructor.php

2. In the Dog class declaration, replace the setValues method with this constructor

```
function __construct
( int $yrs = 2 , int $lbs = 8 , string $fur = ‘black’ )
{
  $this->age = $yrs ;
  $this->weight = $lbs ;
  $this->color = $fur ;
}
```

3. Now, add a simple destructor method

```
function __destruct() { echo '<hr>Object Destroyed.' ; }
```

4. After the Dog class declaration, edit the statement creating the "fido" object – to pass values to its constructor method, in order to initialize properties

```
$fido = new Dog( 3 , 15 , 'brown' ) ;
```

5. Similarly, edit the statement creating the "pooch" object – to pass values to the constructor method

```
$pooch = new Dog( 4 , 18 , 'gray' ) ;
```

The definition of a class method is also known as the method "implementation".

6. Also edit the statement creating the "rover" object – to use default values of the constructor method

```
$rover = new Dog( ) ;
```

7. Delete the statements calling the setValues method of all three objects – the constructor now replaces that method

Although the initial values of the variable members are set by the constructor, setter methods can be added to subsequently change the values – and those new values can be retrieved by the getter methods.

PHP provides several functions to interrogate classes such as **class_exists(*class*)**, **method_exists(*class*, *method*)**, and **property_exists(*class*, *property*)** that return **TRUE** on success. You can also get a list of class members with **get_class_vars(*class*)** and **get_class_methods(*class*)** to see only "visible" members:

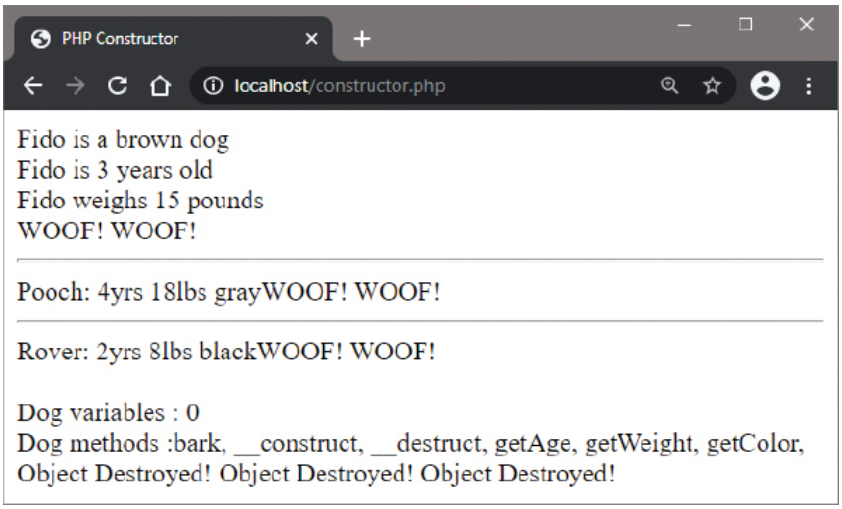
8 Add statements to display the total number of accessible variables in the Dog class

```
$items = get_class_vars( 'Dog' ) ;
echo '<br>Dog variables : '.count( $items ) ;
```

9 Add statements to display the names of accessible methods in the Dog class

```
echo '<br>Dog methods :' ;
$items = get_class_methods( 'Dog' ) ;
foreach( $items as $item ) { echo "$item, " ; }
```

10 Save the document in your web server's **/htdocs** directory as **constructor.php** then open the page via HTTP to see the output set by the constructor



The number of visible variables is zero. Temporarily change their access specifier to public and see it become 3 – encapsulation in action.

Promoting properties

The typical syntax to create a class that declares a property then assigns a value to it in the constructor is quite verbose:

```
class ClassName
{
  access-specifier type $property ;

  function __construct( type $arg )
  {
    $this->property = $arg ;
  }
}
```

Happily, this can be simplified by adding one of the access specifiers (**public**, **private** or **protected**) before parameters of the constructor method. This "promotes" the parameters so that properties of the parameter names will be automatically added to the class, along with a forwarding assignment to those properties in the body of the constructor method. Constructor property promotion is shorthand syntax to declare and assign class properties from the constructor. There is no need to declare the property first, so the syntax now looks like this:

```
class ClassName
{
  function __construct ( access-specifier type $property ) { }
}
```

The ability to promote class properties in the constructor is a great new feature introduced in PHP 8.

The body of the constructor can still contain statements to execute and properties can be accessed using setter and getter methods of the class as usual.

1. Begin a PHP script with a statement to declare a class using standard properties

```
<?php
class Paint_1
{
  private string $color ;

  function construct( string $hue = 'Blue' )
  {
    $this->color = $hue ;
  }

  public function setColor( string $c ){ $this->color = $c ; }
  public function getColor(){ return $this->color ; }
}
```

promoter.php

2. Next, in the script, add a statement to create an instance of the class

```
$pot_1 = new Paint_1() ;
```

3. Now, add statements to call the class methods

```
echo $pot_1->getColor().'<br>' ;
echo $pot_1->setColor( 'Yellow' ) ;
echo $pot_1->getColor().'<hr>' ;
```

4. Then, declare a similar class with simplified syntax using promoted properties

```
class Paint_2
{
  function__construct( private string $color = 'Red' ) { }
  public function setColor( string $c ){ $this->color = $c ; }
  public function getColor(){ return $this->color ; }
}
```

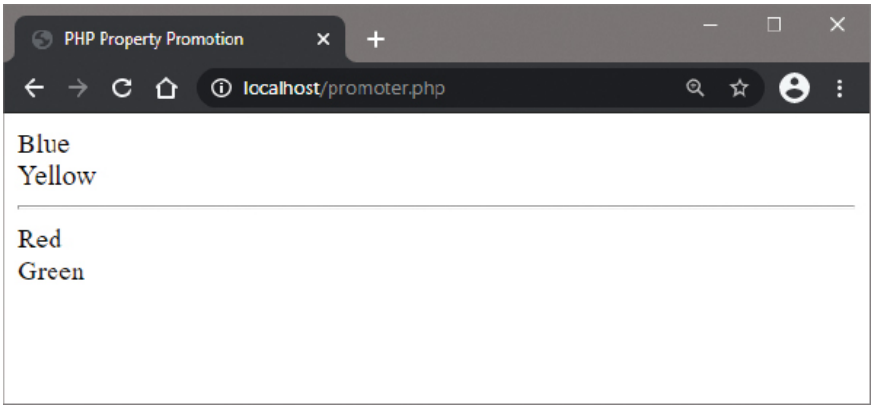
5 Next, in the script, add a statement to create an instance of the similar class

```
$pot_2 = new Paint_2() ;
```

6 Finally, add statements to call the similar class methods

```
echo $pot_2->getColor().'<br>' ;
echo $pot_2->setColor( 'Green' ) ;
echo $pot_2->getColor() ;
?>
```

7 Save the document in your web server's **/htdocs** directory as **promoter.php** then open the page via HTTP to see the output from the standard and promoted properties



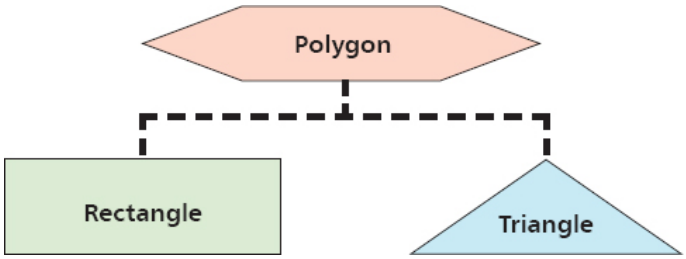
Only the **__construct()** method can have promoted properties.

It is technically possible to mix standard properties and promoted properties but this can produce unclear code – use only one technique.

Inheriting properties

A PHP class can be created as a brand new class, like those in previous examples, or can be “derived” from an existing class. Importantly, a derived class inherits members of the parent (base) class from which it is derived – in addition to its own members.

The ability to inherit members from a base class allows derived classes to be created that share certain common properties, which have been defined in the base class. For example, a “Polygon” base class may define width and height properties that are common to all polygons. Classes of “Rectangle” and Triangle” could be derived from the Polygon class – inheriting width and height properties, in addition to their own members defining their unique features.



The virtue of inheritance is extremely powerful and is the second principle of Object Oriented Programming (OOP).

A derived class declaration adds the **extends** keyword after its class name, followed by the name of the class from which it derives:

1. Begin a PHP script by declaring a base class named “Polygon”, which has two private variables initialized by the constructor, and has two public getter methods

```
<?php
class Polygon
{
  private $width , $height ;

  function __construct( int $w = 4 , int $h = 5 )
  {
```

```
    $this->width = $w ;
    $this->height = $h ;
  }

  public function getWidth() { return $this->width ; }
  public function getHeight() { return $this->height ; }
}
```

inheritance.php

2. After the Polygon class, declare a “Rectangle” class that derives from the Polygon class and adds a unique method

```
class Rectangle extends Polygon
{
  public function area()
  {
    return ( $this->getWidth() * $this->getHeight() ) ;
  }
}
```

3. After the Rectangle class, declare a “Triangle” class that derives from the Polygon class and adds a unique method

```
class Triangle extends Polygon
{
  public function area()
  {
    return ( $this->getWidth() * $this->getHeight() / 2 ) ;
  }
}
```

4. After the Triangle class, add two statements creating an instance of each derived class using default values

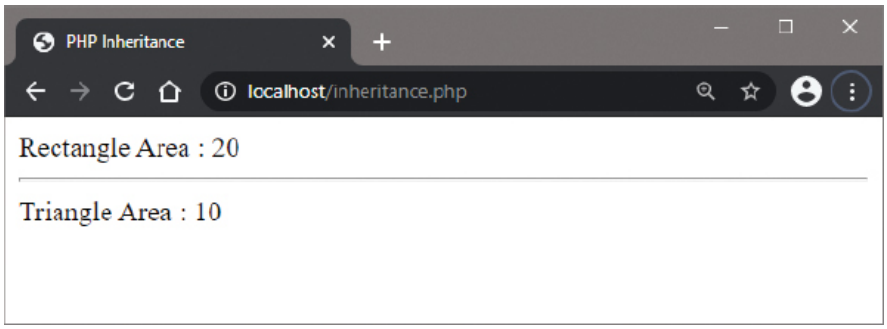
```
$rect = new Rectangle() ;
```

```
$trio = new Triangle() ;
```

5 Finally, add two statements to output the value returned by the unique method of each derived class

```
echo 'Rectangle Area : '.$rect->area().'<hr>' ;
echo 'Triangle Area : '.$trio->area() ;
?>
```

6 Save the document in your web server's **/htdocs** directory as **inheritance.php** then open the page via HTTP to see the output from inherited members



Don't confuse class instances and derived classes – an instance is a copy of a class, whereas a derived class is a new class that inherits properties of the base class from which it is derived.

In PHP, a derived class is always dependent on a single base class – multiple inheritance is not supported.

Embracing polymorphism

The three cornerstones of Object Oriented Programming (OOP) are encapsulation, inheritance, and polymorphism. On the previous pages we have seen how data can be encapsulated within a PHP class, and how derived classes inherit the properties of their base class, so we can now explore the final cornerstone principle of polymorphism.

The term “polymorphism” (from Greek, meaning “many forms”) describes the ability to assign a different meaning or purpose to an entity according to its context. For example, in the real world everyone knows how to use a push button (entity) – you simply apply pressure to it. What that button does, however, depends on what is connected to it (context), but the result does not affect how that button is used.

In the world of Object Oriented Programming, polymorphism allows classes to have different functionality while sharing a common interface. The interface does not need to know which class it is using because they are, like the real world button, all used the same way, but can produce different contextual results. Embracing polymorphism in this way enables the programmer to create interchangeable objects that can be selected according to requirement, rather than by conditional tests.

A PHP interface is similar to a class but is declared using the **interface** keyword and can only contain method definitions, not any method statements to be executed, like this:

```
interface InterfaceName
{
  // Method definitions only go here.
}
```

Method definitions in a interface declaration may also specify parameters.

Each class that implements an interface must implement all the methods it defines by specifying their statements to be executed. A class is attached to an interface by including the **implements** keyword and interface name in its declaration, like this:

```
class ClassName implements InterfaceName
{
  // Method implementations go here.
}
```

The previous example can be modified to embrace polymorphism by adding an interface and a single function.

1. Rename a copy of the previous example “inheritance.php” as a new program “polymorphism.php”

polymorphism.php

2. Declare an interface named “Shape”, which contains a definition for the methods already implemented in the Rectangle and Triangle classes

```
interface Shape { public function area() ; }
```

3. Edit the first line of the Rectangle and Triangle class declarations – to associate them with the interface

```
class Rectangle extends Polygon implements Shape
class Triangle extends Polygon implements Shape
```

4. After the class blocks, add a function that accepts an argument of the Shape type and calls its defined method

```
function getArea( Shape $shape )
{ return $shape->area() ; }
```

5. Edit the two statements creating an instance of each derived class – to pass in argument values

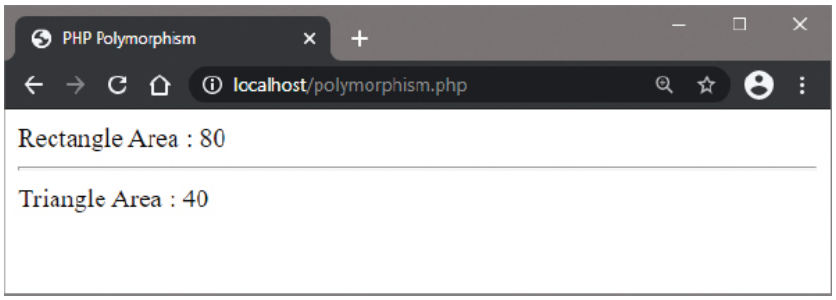
```
$rect = new Rectangle( 8 , 10 ) ;
```

```
$trio = new Triangle( 8 , 10 ) ;
```

6 Finally, edit the two statements to output the value returned by the unique method of each derived class, by calling each method via the interface

```
echo 'Rectangle Area : '.getArea( $rect ).'<hr>' ;
echo 'Triangle Area : '.getArea( $trio ) ;
```

7 Save the document in your web server's **/htdocs** directory as **polymorphism.php** then open the page via HTTP to see the output via a polymorphic interface



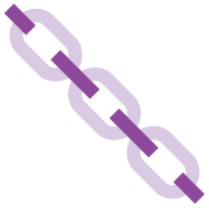
What's going on here?

Each call to the new **getArea()** function passes an instance object as an argument.

The **getArea()** function calls the **area()** method defined in the interface.

The output is returned from the implementation of the **area()** method of the associated instance.

Chaining method calls



You may often want to call a method, or get a property, only if the result of one or more tested expressions are not **null**. This can quickly lead to deeply nested conditional tests, like these:

```
$country = null ;

if( $session !== null )
{
  $user = $session->user ;
  if ( $user !== null )
  {
    $address = $user->getAddress() ;
    if( $address !== null )
    {
          $country = $address->country ;
    }
  }
}
```

The **?->** nullsafe operator is a great new feature introduced in PHP 8.

The same series of conditional tests can be made more concisely in a single chain using the PHP **?->** “nullsafe” operator, like this:

```
$country = $session?->user?->getAddress()?->country ;
```

The chain is evaluated from left to right. When the item at the left of the nullsafe operator evaluates to **null**, the execution of the entire chain stops immediately and the chain evaluates to **null**.

1 Begin a PHP script by declaring a class that assigns a **null** value to a property

```
class Session
{
  public $user ;
  function construct() { $this->user = null ; }
}
```

nullsafe.php

2 Declare a class containing a method to return an object

```
class User
{
  public function getAddress() { return new Address() ; }
}
```

3 Declare a class that assigns a **string** value to a property

```
class Address
{
  public $country ;
  function construct() { $this->country = 'USA' ; }
}
```

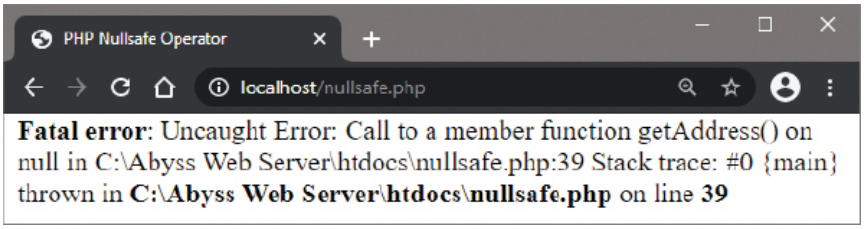
4 Then, create an instance of the class declared in Step 1

```
$session = new Session() ;
```

5 Finally, add a chain statement without nullsafe operators that attempts to output the **string** property

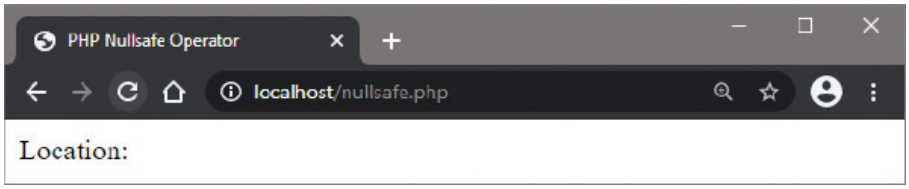
```
$country = $session->user->getAddress()->country ;
echo 'Location: '.$country ;
```

6 Save the document in your web server's **/htdocs** directory as **nullsafe.php** then open the page via HTTP to see the attempt fail with a fatal error message



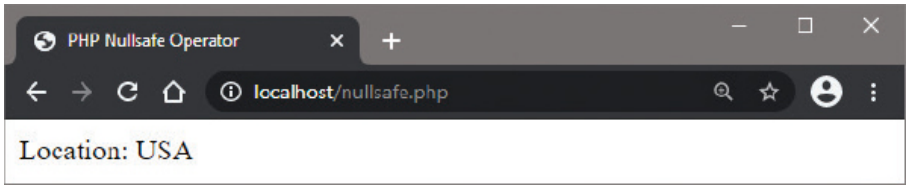
7 Edit the chain statement in Step 5 to add nullsafe operators, then save the script and refresh the browser to see the attempt fail gracefully

```
$country = $session?->user?->getAddress()?->country ;
```



8 Edit the assignment in Step 1 to assign an object to the property, then save the script once more and refresh the browser to see the attempt succeed

```
function__construct() { $this->user = new User() ; }
```



Skipping evaluation of the remainder of the chain when an item in a chain evaluates to **null** is known as "short circuiting". When the item at the left hand side of the nullsafe operator does not evaluate to **null**, execution continues to evaluate the next item in the chain until the end of the chain is reached.

Summary

- A PHP class is a data structure that can contain both variables and functions in a single entity.
- Classes are declared using the **class** keyword and can contain member variable properties and member function methods.
- Access to class members from outside the class is controlled by the **public**, **private** or **protected** access specifier keywords.
- Variable properties are encapsulated by declaring them **private** so their values can only be accessed by **public** methods.
- In order to create an object you must declare an instance of a **class** data structure using the **new** keyword.
- The special **->** object operator can be used to reference any visible class member.
- Setter and getter methods can reference a member of the same class using the **$this** pseudo-variable.
- The **__construct()** constructor method is called when a class is created and can be used to initialize variable properties.
- Adding **public**, **private** or **protected** access specifiers for constructor parameters can promote properties to the class.
- The **__destruct()** destructor method is called when a class is destroyed and can be used to execute final statements.
- PHP has functions to interrogate classes, listing methods with **get_class_methods()** and variables with **get_class_vars().**
- The three cornerstones of Object Oriented Programming are encapsulation, inheritance, and polymorphism.
- A derived class declaration will include the **extends** keyword to specify a base class from which to inherit members.
- An interface is declared using the **interface** keyword and the declaration may contain only method definitions.

- A class is attached to an interface by including the **implements** keyword and interface name in its declaration.
- Conditional tests for null values can be made concisely in a single chain using the PHP **?->** “nullsafe” operator.